

Foundations of YOLO object detection

[YOLO object detection]

1.0 Content Block

YOLOv1 The original YOLO algorithm, introduced in 2015, divides the image into an $S \times S$ grid of cells. If the center of an object's bounding box falls into a grid cell, that cell is said to "contain" that object. Each grid cell predicts B bounding boxes and confidence scores for those boxes. These confidence scores reflect how confident the model is that the box contains an object and how accurate it thinks the box is that it predicts. In more detail, the network performs the same convolutional operation over each of the S^2 patches. The output of the network on each patch is a tuple as follows: $(p_1, \dots, p_C, c_1, x_1, y_1, w_1, h_1, \dots, c_B, x_B, y_B, w_B, h_B)$ where

2.0 Content Block

You Only Look Once (YOLO) is a series of real-time object detection systems based on convolutional neural networks. First introduced by Joseph Redmon et al. in 2015, YOLO has undergone several iterations and improvements, becoming one of the most popular object detection frameworks. The name "You Only Look Once" refers to the fact that the algorithm requires only one forward propagation pass through the neural network to make predictions, unlike previous region proposal-based techniques like R-CNN that require thousands for a single image.

3.0 Content Block

Overview Compared to previous methods like R-CNN and OverFeat, instead of applying the model to an image at multiple locations and scales, YOLO applies a single neural network to the full image. This network divides the image into regions and predicts bounding boxes and probabilities for each region. These bounding boxes are weighted by the predicted probabilities.

4.0 Content Block

c_j is the predicted intersection over union (IoU) of each bounding box with its corresponding ground truth. The network architecture has 24 convolutional layers followed by 2 fully connected layers. During training, for each cell, if it contains a ground truth bounding box, then only the predicted bounding boxes with the highest IoU with the ground truth bounding boxes is used for gradient descent. Concretely, let j be that predicted bounding box, and let i be the ground truth class label, then x_j, y_j, w_j, h_j are trained by gradient descent to approach the ground truth, p_i is trained towards 1, other $p_{i'}$ are trained towards zero. If a cell contains no ground truth, then only $c_1, c_2, \dots, c_B$ are trained by gradient descent to approach zero.

5.0 Content Block

YOLOv4 and beyond Subsequent versions of YOLO (v4, v5, etc.) have been developed by different researchers, further improving performance and introducing new features. These versions are not officially associated with the original YOLO authors but build upon their work. As of 2025, there are versions up to YOLOv12.

6.0 Content Block

YOLOv2 Released in 2016, YOLOv2 (also known as YOLO9000) improved upon the original model by incorporating batch normalization, a higher resolution classifier, and using anchor boxes to predict

bounding boxes. It could detect over 9000 object categories. It was also released on GitHub under the Apache 2.0 license.

7.0 Content Block

x_j, y_j, w_j, h_j are the center coordinates, width, and height of the j -th predicted bounding box that is centered in the cell. Multiple bounding boxes are predicted to allow each prediction to specialize in one kind of bounding box. For example, slender objects might be predicted by $j = 2$ while stout objects might be predicted by $j = 1$.

8.0 Content Block

Train a neural network for image classification only ("classification-trained network"). This could be one like the AlexNet. The last layer of the trained network is removed, and for every possible object class, initialize a network module at the last layer ("regression network"). The base network has its parameters frozen. The regression network is trained to predict the (x, y) coordinates of two corners of the object's bounding box. During inference time, the classification-trained network is run over the same image over many different zoom levels and croppings. For each, it outputs a class label and a probability for that class label. Each output is then processed by the regression network of the corresponding class. This results in thousands of bounding boxes with class labels and probability. These boxes are merged until only one single box with a single class label remains.